MyCodeSchool

React + TypeScript — exerciții de acomodare

10 demo-uri · 50 de exerciții

Pornire: în folderul proiectului rulezi `npm install` (o singură dată), apoi `npm run dev` și deschizi `http://localhost:5173`.

Cum lucrezi: citești noțiunea în `TEORIE.pdf`, apoi în meniul din stânga alegi demo-ul ei, îl citești în `src/demos/`, apoi rezolvi exercițiile lui în `src/exercitii/`. Fiecare fișier de exercițiu există deja; îi înlocuiești conținutul. Pune tip pe fiecare variabilă, parametru și funcție.

1a. Profil — `Ex1aProfile.tsx`

Teorie: Demo 1

1. Componenta întoarce un fragment `<>...</>`.
2. În el: un `<h1>` cu numele tău, un `<p>` cu orașul și o listă `<ul>` cu 3 hobby-uri.

Pe ecran

```
Ion Popescu
Oraș: Cluj-Napoca
• fotbal  • muzică  • programare
```

1b. Carte de vizită — `Ex1bCard.tsx`

Teorie: Demo 1

1. Componenta întoarce un singur `<div className="card">` (clasa există deja în `index.css`).
2. În el: un `<h2>` cu numele și două `<p>`: emailul și telefonul.

Indiciu: fără `className` cardul apare fără chenar; `class` nu merge în JSX.

1c. Rețetă — `Ex1cRecipe.tsx`

Teorie: Demo 1

1. Un `<h1>` cu numele rețetei.
2. Un `<h2>` „Ingrediente” urmat de o listă `<ul>` cu 4 ingrediente.
3. Un `<h2>` „Pași” urmat de o listă numerotată `<ol>` cu 3 pași.

1d. Orar — `Ex1dSchedule.tsx`

Teorie: Demo 1

1. Un `<table>` cu `<thead>` (coloanele Zi și Materie) și `<tbody>` cu 3 rânduri.

Indiciu: În JSX, `<tr>` stă obligatoriu în `<thead>` sau `<tbody>`; altfel React afișează un avertisment în consolă.

1e. Meniu restaurant — `Ex1eMenu.tsx`

Teorie: Demo 1

1. Un `<h1>` „Meniu”.
2. Două `<div className="card">`, fiecare cu un `<h3>` (preparatul) și un `<p>` (prețul).

2a. Despre mine — `Ex2aAbout.tsx`

Teorie: Demo 2

1. Trei constante cu tip: `firstName: string`, `lastName: string`, `birthYear: number`.
2. O funcție `ageIn(year: number): number` care întoarce `year - birthYear`.
3. Afișezi numele complet într-un `<h1>`, apoi vârsta în 2026 și în 2030, chemând `ageIn` în `{ }`.

Pe ecran (birthYear = 2005)

```
Ion Popescu
Născut în 2005
În 2026 are 21 ani
În 2030 va avea 25 ani
```

2b. Dreptunghi — `Ex2bRectangle.tsx`

Teorie: Demo 2

1. Constantele `width: number = 5` și `height: number = 3`.
2. Funcțiile `area(w: number, h: number): number` și `perimeter(w: number, h: number): number`.
3. Afișezi rezultatele lor.

Pe ecran

```
Dreptunghi 5 × 3
Aria: 15
Perimetrul: 16
```

2c. Temperatură — `Ex2cTemperature.tsx`

Teorie: Demo 2

1. `const celsius: number = 25;`
2. Funcția `toFahrenheit(c: number): number` întoarce `c * 9 / 5 + 32`.
3. Afișezi conversia pentru `celsius`, pentru 0 și pentru 100.

Pe ecran

```
25°C = 77°F
0°C = 32°F
100°C = 212°F
```

2d. Preț cu TVA — `Ex2dVat.tsx`

Teorie: Demo 2

1. Constantele `product: string`, `price: number = 100`, `vat: number = 0.19`.
2. Funcția `priceWithVat(p: number): number`.
3. Afișezi produsul, prețul fără TVA, valoarea TVA și prețul cu TVA.

Pe ecran

```
Căști
Preț fără TVA: 100 lei
TVA: 19 lei
Preț cu TVA: 119 lei
```

2e. Inițiale și majorat — `Ex2eInitials.tsx`

Teorie: Demo 2

1. Constantele `firstName`, `lastName` (string) și `age: number = 17`.
2. Funcțiile `initials(first: string, last: string): string` și `isAdult(a: number): boolean`.
3. Afișezi inițialele în `<h1>`, apoi `isAdult(age)` și `isAdult(age + 1)`.

Pe ecran

```
MI
Maria Ionescu, 17 ani
Major: false
Major peste un an: true
```

Indiciu: Un `boolean` nu apare pe ecran în JSX; îl afișezi cu `.toString()`.

3a. Butoane în consolă — `Ex3aConsole.tsx`

Teorie: Demo 3

1. Funcția `sayHello(): void` scrie în consolă `Salut din React!`.
2. Funcția `showTime(): void` scrie ora curentă.
3. Două butoane: „Spune salut” și „Arată ora”, legate cu `onClick`.

Indiciu: `new Date().toLocaleTimeString()` dă ora. Scrii `onClick={sayHello}`, fără `()`: altfel funcția rulează imediat, nu la click.

3b. Plus / minus — `Ex3bPlusMinus.tsx`

Teorie: Demo 3

1. `let points: number = 10;` și un `<h1>` care afișează `Puncte: {points}`.
2. Butoanele „+” și „–” cresc și scad `points` și îl scriu în consolă.
3. Sub butoane, într-un `<p>`, scrii cu cuvintele tale de ce pe ecran rămâne 10.

3c. Alege o culoare — `Ex3cColors.tsx`

Teorie: Demo 3

1. Funcția `choose(color: string): void` scrie în consolă `Ai ales: <culoare>`.
2. Trei butoane: Roșu, Verde, Albastru; fiecare cheamă `choose` cu altă culoare.

Indiciu: Când funcția primește un argument, folosești o funcție săgeată: `onClick={() => choose("Roșu")}`.

3d. Calculator în consolă — `Ex3dCalculator.tsx`

Teorie: Demo 3

1. `const a: number = 8;` și `const b: number = 2;`, afișate într-un `<h1>`.
2. Funcția `show(operation: string, result: number): void` scrie în consolă, de exemplu `8 + 2 = 10`.
3. Patru butoane: +, −, ×, ÷.

3e. Zar în consolă — `Ex3eDice.tsx`

Teorie: Demo 3

1. Funcția `roll(): number` întoarce un număr între 1 și 6.
2. Butonul „Aruncă zarul” scrie în consolă `Zarul arată: 4`.

Indiciu: `Math.floor(Math.random() * 6) + 1`. Rezultatul nu poate apărea încă pe ecran: pentru asta ai nevoie de stare (Demo 4).

4a. Contor — `Ex4aCounter.tsx`

Teorie: Demo 4

1. `const [count, setCount] = useState<number>(0);`
2. Butoanele „−1”, „+1” și „Reset”; numărul apare într-un `<h1>`.
3. Contorul nu coboară sub 0.

Indiciu: în `decrement()` chemi `setCount` doar dacă `count > 0`.

4b. Voturi — `Ex4bVotes.tsx`

Teorie: Demo 4

1. Două stări de tip `number`: `likes` și `dislikes`.
2. Butonul „👍 {likes}” crește likes, butonul „👎 {dislikes}” crește dislikes.
3. Sus afișezi `Scor: {likes − dislikes}`.

Pe ecran după 3 👍 și 1 👎

```
Scor: 2      [👍 3]  [👎 1]
```

4c. Zar pe ecran — `Ex4cDice.tsx`

Teorie: Demo 4

1. Două stări: `value` (pornește 1) și `rolls` (pornește 0), ambele `number`.
2. Butonul „Aruncă zarul” pune un număr aleator în `value` și crește `rolls`.
3. Afișezi `🎲 {value}` și `Aruncări: {rolls}`.

4d. Pași +1 / +5 / +10 — `Ex4dStep.tsx`

Teorie: Demo 4

1. O stare `count: number`.
2. Funcția `add(step: number): void` adună `step` la `count`.
3. Butoanele +1, +5, +10 și Reset.

Pe ecran

```
după +1, +5, +10:  16
```

4e. Semafor — `Ex4eTrafficLight.tsx`

Teorie: Demo 4

1. O stare `color: string`, pornește `"gray"`.
2. Butoanele Roșu, Galben, Verde pun `"red"`, `"orange"`, `"green"`.
3. Titlul își schimbă culoarea: `<h1 style={{ color: color }}>`.

Indiciu: La `style` sunt două acolade: prima intră în JavaScript, a doua e obiectul cu proprietățile CSS.

5a. Numără caractere — `Ex5aCharCount.tsx`

Teorie: Demo 5

1. O stare `text` de tip `string`, legată de un `<input>` prin `value` și `onChange`.
2. Sub input: `{text.length} / 50 caractere` și câte caractere mai poate scrie.

Pe ecran după „Salut”

```
5 / 50 caractere
Mai poți scrie 45 caractere.
```

5b. Nume complet — `Ex5bFullName.tsx`

Teorie: Demo 5

1. Două stări `string`: `firstName` și `lastName`, fiecare cu input-ul ei.
2. Afișezi numele complet și inițialele.

Pe ecran pentru Ion / Popescu

```
Nume complet: Ion Popescu
Inițiale: IP
```

Indiciu: `firstName.charAt(0)` dă prima literă.

5c. Oglindă — `Ex5cMirror.tsx`

Teorie: Demo 5

1. O stare `text: string` legată de un input.
2. Afișezi textul cu litere mari, cu litere mici și scris invers.

Pe ecran

```
pentru „React”:
Litere mari: REACT
Litere mici: react
Invers: tcaeR
```

Indiciu: `text.split("").reverse().join("")` întoarce textul invers.

5d. Anul nașterii — `Ex5dBirthYear.tsx`

Teorie: Demo 5 + Demo 6

1. O stare `year: number`, pornește 2000.
2. Un `<input type="number">`; la `onChange` transformi textul în număr.
3. Afișezi `În 2026 ai {2026 - year} ani.`

Indiciu: `setYear(Number(e.target.value))`, la fel ca în Demo 6.

5e. Salut personalizat — `Ex5eGreeting.tsx`

Teorie: Demo 5

1. Două stări `string`: `name` (input-ul) și `greeting` (mesajul).
2. Butonul „Salută” pune în `greeting` textul `Salut, <name>!` și golește input-ul.
3. Mesajul rămâne pe ecran și după ce input-ul s-a golit.

6a. Parolă — `Ex6aPassword.tsx`

Teorie: Demo 6

1. Un `<input type="password">` legat de starea `password: string`.
2. Cu `? :`: dacă are sub 8 caractere afișezi cu roșu (`className="red"`) „Prea scurtă: minim 8 caractere”, altfel cu verde (`"green"`) „Parolă OK”.

6b. Mesaj secret — `Ex6bSecret.tsx`

Teorie: Demo 6

1. O stare `visible` de tip `boolean`, pornește `false`.
2. Butonul schimbă `visible` în opusul lui; textul butonului e „Arată” sau „Ascunde”.
3. Cu `&&`: „Secretul este 42.” apare doar când `visible` e `true`.

Indiciu: `setVisible(!visible)`.

6c. Par sau impar — `Ex6cEvenOdd.tsx`

Teorie: Demo 6

1. O stare `n: number` legată de un `<input type="number">`.
2. Cu `? :` afișezi „Par” sau „Impar”.
3. Cu `&&` afișezi cu roșu „Număr negativ” doar când `n < 0`.

Indiciu: Un număr e par când `n % 2 === 0`.

6d. Admis / respins — `Ex6dGrade.tsx`

Teorie: Demo 6

1. O stare `grade: number` legată de un input numeric.
2. Cu `? :`: de la 5 în sus „Admis” cu verde, altfel „Respins” cu roșu.
3. Cu `&&`: la nota 10 apare și „Felicitări!”.

6e. Login — `Ex6eLogin.tsx`

Teorie: Demo 6

1. Stările `username`, `password` (string), `loggedIn` și `error` (boolean).
2. Funcția `login(): void`: dacă utilizatorul e `admin` și parola `1234`, `loggedIn` devine `true`; altfel `error` devine `true`.
3. Cu `? :`: logat → „Bine ai venit, admin!”; nelogat → cele două input-uri și butonul „Intră”.
4. Cu `&&`: „Utilizator sau parolă greșită” cu roșu, doar când `error` e `true`.

7a. Filme — `Ex7aMovies.tsx`

Teorie: Demo 7

1. Deasupra componentei: `type Movie = { id: number; title: string; year: number };`
2. În componentă: `const movies: Movie[]` cu 4 filme.
3. Titlul `Filme (4)` și o listă `<li>` cu „titlu (an)”, făcută cu `map`, cu `key={movie.id}`.

7b. Șterge filme — `Ex7bDelete.tsx`

Teorie: Demo 4 + Demo 7

1. Aceleași 4 filme, de data asta în stare: `useState<Movie[]>([...])`.
2. Lângă fiecare film, un buton „Șterge” care cheamă `remove(id: number): void`.
3. Titlul arată câte filme au rămas; când lista e goală apare „Niciun film.”.

Indiciu: `movies.filter((movie) => movie.id !== id)` întoarce lista fără filmul șters; o dai lui `setMovies`.

7c. Cumpărături — `Ex7cShopping.tsx`

Teorie: Demo 7

1. `const items: string[]` cu 5 produse.
2. Titlul `Cumpărături (5)` și lista făcută cu `map`.

Indiciu: Când lista e de texte diferite între ele, textul poate fi chiar cheia: `key={item}`.

7d. Doar adulții — `Ex7dAdults.tsx`

Teorie: Demo 7 + Demo 10

1. `type Person = { id: number; name: string; age: number };` și 4 persoane, dintre care 2 minore.
2. `const adults: Person[] = people.filter(...)` păstrează doar persoanele de la 18 ani în sus.
3. Titlul `Adulți: 2 din 4` și lista adulților cu `map`.

7e. Listă de sarcini — `Ex7eTodo.tsx`

Teorie: Demo 4 + Demo 7

1. `type Task = { id: number; title: string };`
2. Stările `tasks: Task[]` (pornește `[]`) și `title: string` (input-ul).
3. Butonul „Adaugă” pune o sarcină nouă la sfârșitul listei și golește input-ul.
4. Titlul arată câte sarcini sunt; când lista e goală apare „Nicio sarcină.”.

Indiciu: `setTasks([...tasks, { id: Date.now(), title: title }])`: `...tasks` copiază lista veche, apoi adaugi elementul nou.

8a. Note în cuvinte — `Ex8aGradeLabel.tsx`

Teorie: Demo 8

1. Funcția `label(grade: number): string` cu `if / else if / else`: 10 → „Excelent”, de la 8 → „Foarte bine”, de la 5 → „Suficient”, altfel „Insuficient”.
2. Afișezi rezultatul pentru 4, 6, 9 și 10.

Pe ecran

```
4: Insuficient
6: Suficient
9: Foarte bine
10: Excelent
```

Indiciu: Ordinea contează: verifici întâi 10, apoi 8, apoi 5.

8b. Anotimp — `Ex8bSeason.tsx`

Teorie: Demo 8

1. O stare `month: number` legată de un input numeric.
2. Funcția `season(m: number): string`: 12, 1, 2 → Iarnă; 3–5 → Primăvară; 6–8 → Vară; 9–11 → Toamnă; altceva → „Lună invalidă”.
3. Afișezi `Luna 7: Vară`.

Indiciu: Prima condiție e cea pentru lună invalidă: `m < 1 || m > 12`.

8c. Bilet de film — `Ex8cTicket.tsx`

Teorie: Demo 8

1. O stare `age: number` legată de un input numeric.
2. Funcția `ticketPrice(a: number): number`: sub 7 ani 0 lei, sub 18 ani 10 lei, de la 65 de ani 15 lei, altfel 25 lei.
3. Afișezi `Preț: X lei`.

8d. Cel mai mare — `Ex8dMax.tsx`

Teorie: Demo 8

1. Funcția `max3(a: number, b: number, c: number): number`, scrisă cu `if`, fără `Math.max`.
2. Afișezi rezultatul pentru (3, 9, 5), (12, 4, 7) și (-1, -8, -3).

Pe ecran

```
max(3, 9, 5) = 9
max(12, 4, 7) = 12
max(−1, −8, −3) = −1
```

Indiciu: Pornești cu `let max: number = a;` și îl înlocuiești dacă `b` sau `c` e mai mare.

8e. Validare email — `Ex8eEmail.tsx`

Teorie: Demo 8

1. O stare `email: string` legată de un input.
2. Funcția `validate(value: string): string`: gol → „Emailul este obligatoriu”; fără `@` → „Lipsește @”; fără punct → „Lipsește domeniul (ex. .ro)”; altfel „Email valid”.
3. Mesajul se schimbă pe măsură ce scrii.

Indiciu: `value.includes("@")` întoarce `true` sau `false`; `!` îl inversează.

9a. Suma 1..n — `Ex9aSum.tsx`

Teorie: Demo 9

1. O stare `n: number` legată de un input numeric.
2. Funcția `sumTo(limit: number): number` adună cu un `for` numerele de la 1 la `limit`.

Pe ecran

```
1 + 2 + ... + 100 = 5050
```

9b. Tabla înmulțirii — `Ex9bTimesTable.tsx`

Teorie: Demo 9 + Demo 7

1. O stare `n: number`, pornește 7.
2. Funcția `table(x: number): string[]` construiește cu un `for` cele 10 rânduri (`7 × 1 = 7` …).
3. Afișezi rândurile într-un `<ul>` cu `map`.

Indiciu: Pornești cu `const lines: string[] = [];` și adaugi cu `lines.push(...)`.

9c. Numere pare — `Ex9cEvens.tsx`

Teorie: Demo 9

1. Funcția `evensUpTo(limit: number): number[]`: un `for` de la 1 la `limit` și un `if` care păstrează doar numerele pare.
2. Afișezi rezultatul pentru 10 și 25, cu `.join(", ")`.

Pe ecran

```
Până la 10: 2, 4, 6, 8, 10
```

9d. Factorial — `Ex9dFactorial.tsx`

Teorie: Demo 9

1. O stare `n: number` legată de un input numeric.
2. Funcția `factorial(x: number): number` înmulțește cu un `for` numerele de la 2 la `x`.

Pe ecran

```
6! = 720
```

Indiciu: Rezultatul pornește de la 1, nu de la 0.

9e. Vocale — `Ex9eVowels.tsx`

Teorie: Demo 9

1. O stare `text: string` legată de un input.
2. Funcția `countVowels(value: string): number` parcurge literele cu `for (const letter of ...)` și numără vocalele.

Pe ecran

```
„Programare în React” → Vocale: 7
```

Indiciu: `"aeiouăâî".includes(letter)` îți spune dacă litera e vocală; transformi întâi textul cu `toLowerCase()`.

10a. Prețuri cu TVA — `Ex10aVatPrices.tsx`

Teorie: Demo 10

1. `const prices: number[] = [10, 25, 40, 100];`
2. Cu `map` obții `withVat: number[]`, cu prețurile înmulțite cu 1.19 și rotunjite.

Pe ecran

```
Fără TVA: 10, 25, 40, 100
Cu TVA: 12, 30, 48, 119
```

Indiciu: `Math.round(price * 1.19)`.

10b. Cuvinte lungi — `Ex10bLongWords.tsx`

Teorie: Demo 10

1. `const words: string[]` cu 6 cuvinte.
2. Cu `filter` păstrezi cuvintele cu peste 5 litere.
3. Afișezi toate cuvintele, cuvintele lungi și câte sunt.

10c. Caută produs — `Ex10cFindProduct.tsx`

Teorie: Demo 10 + Demo 6

1. `type Product = { id: number; name: string; price: number };` și 3 produse.
2. O stare `search: string` legată de un input.
3. Cu `find` cauți produsul cu numele scris; cu `? :` afișezi „mouse: 80 lei” sau „Produsul nu există”.

Indiciu: `find` întoarce `Product | undefined`; compari cu `search.toLowerCase()`, ca să meargă și „Mouse”.

10d. Total coș — `Ex10dCart.tsx`

Teorie: Demo 10 + Demo 7

1. `type CartItem = { id: number; name: string; price: number; quantity: number };` și 3 produse.
2. Lista cu `map`: `Caiet: 4 × 5 lei = 20 lei`.
3. Totalul cu `reduce`.

Pe ecran

```
Total: 170 lei
```

Indiciu: `cart.reduce((sum, item) => sum + item.price * item.quantity, 0)`.

10e. Filtru orașe — `Ex10eCityFilter.tsx`

Teorie: Demo 10 + Demo 5

1. `const cities: string[]` cu 6 orașe.
2. O stare `text: string` legată de un input.
3. Cu `filter` + `includes` păstrezi orașele care conțin textul scris; le afișezi cu `map` și numărul lor în titlu.
4. Când nu rămâne niciunul: „Niciun oraș găsit.”.

Indiciu: Pui și orașul, și textul cu `toLowerCase()`, ca să nu conteze literele mari.